

学习笔记

项目简介

vue3-element-admin 基于 Vue3、Vite、TypeScript 和 Element-Plus 搭建的极简开箱即用企业级后台管理前端模板。配套 Java 后端 youlai-boot、多租户后端 youlai-boot-tenant 和 Node 后端 youlai-nest。提供开发简版vue3-element-template 和 JS 版本vue3-element-admin-js 供开发者快速开发。

项目特色

- 简洁易用**：基于 vue-element-admin 升级的 Vue3 版本，无过渡封装，易上手。
- 数据交互**：支持 Mock 数据和线上接口文档，并提供配套的 Java 和 Node 后端源码。
- 系统功能**：提供用户管理、角色管理、菜单管理、部门管理、字典管理、系统配置、通知公告等功能模块。
- 权限管理**：支持动态路由、按钮权限、角色权限和数据权限等多种权限管理方式。
- 多租户**：支持多租户模式与租户隔离。
- 基础设施**：提供国际化、多布局、暗黑模式、全屏、水印、接口文档和代码生成器等功能。
- 持续更新**：项目持续开源更新，实时更新工具和依赖。

项目代码仓库：<https://gitee.com/youlaiorg/vue3-element-admin>
项目在线文档：<https://www.youlai.tech/docs/admin/>

项目启动

环境准备

环境类型	版本要求	备注
Node.js	<code>^20.19.0</code> 或 <code>>=22.12.0</code>	推荐使用 LTS 版本（主版本为偶数）
包管理器	<code>pnpm >= 8.0.0</code>	项目使用 pnpm 作为包管理器
开发工具	Visual Studio Code	推荐安装 Vue、TypeScript 相关插件

快速开始

```
● ● ●
# 克隆代码
git clone https://gitee.com/youlaiorg/vue3-element-admin.git

# 切换目录
cd vue3-element-admin

# 安装 pnpm
npm install pnpm -g

# 设置镜像源(可忽略)
pnpm config set registry https://registry.npmmirror.com

# 安装依赖
pnpm install

# 启动运行
pnpm run dev
```

项目部署

执行 `pnpm run build` 命令后，项目将被打包并生成 `dist` 目录。接下来，将 `dist` 目录下的文件上传到服务器 `/usr/share/nginx/html` 目录下，并配置 Nginx 进行反向代理。

```
● ● ●
pnpm run build
```

项目结构

本文档介绍 vue3-element-admin 的目录结构设计理念和组织方式。

架构选型

Vue 3 项目常见的架构模式有：

架构	适用场景	优点	缺点
扁平结构	小型项目、PoC	简单直接	不可扩展
原子设计	组件库	组件复用性高	层级复杂
模块化	中大型应用	松耦合、可维护	可能重复
Feature-Sliced	大型长期项目	高内聚、边界清晰	初期复杂
微前端	多团队协作	独立部署	协调成本高

本项目采用**模块化架构**，这是 Vue 3 应用最适合的架构方式：

- 按功能模块划分目录
- 模块间松耦合
- 便于独立开发和维护
- 可平滑演进到微前端

参考：[Vue FAQ - Project Structure](#)

目录结构

vue3-element-admin/

|

├─ mock/

|

├─ public/

|

│ └─ favicon.ico

|

├─ src/

|

│ ├─ api/

|

│ │ ├─ system/

|

│ │ │ ├─ user.ts

|

│ │ │ ├─ role.ts

|

│ │ │ └─ menu.ts

|

│ │ ├─ auth.ts

|

│ │ └─ file.ts

|

│ ├─ assets/

|

│ │ ├─ icons/

|

│ │ └─ images/

|

│ ├─ components/

|

│ │ ├─ AiAssistant/

|

│ │ ├─ Breadcrumb/

|

│ │ ├─ CommandPalette/

|

│ │ ├─ CopyButton/

|

│ │ ├─ CURD/

|

│ │ ├─ DictSelect/

|

│ │ ├─ DictTag/

|

│ │ ├─ ECharts/

|

│ │ ├─ Fullscreen/

|

│ │ ├─ Hamburger/

|

│ │ ├─ IconSelect/

|

│ │ ├─ InputTag/

|

│ │ ├─ LangSelect/

|

│ │ ├─ NoticeDropdown/

|

│ │ ├─ OperationColumn/

|

│ │ ├─ Pagination/

|

│ │ ├─ SizeSelect/

|

│ │ ├─ TableSelect/

|

│ │ ├─ TenantSwitcher/

|

│ │ ├─ TextScroll/

|

│ │ ├─ ThemeSwitch/

|

│ │ ├─ Upload/

|

│ │ └─ WangEditor/

|

│ ├─ composables/

|

│ └─ ai/

|

└─

Mock 数据

静态资源（不经过构建）

API 接口层

系统管理模块

认证接口

文件接口

静态资源（经过构建）

SVG 图标

图片资源

公共组件

AI 助手

面包屑

命令面板

复制按钮

CURD 快速开发组件

字典选择器

字典标签

图表组件

全屏切换

菜单折叠

图标选择器

标签输入

语言选择

通知下拉

操作列

分页组件

尺寸选择

表格选择器

租户切换

文字滚动

主题切换

上传组件

富文本编辑器

组合式函数

AI 相关

└─ useAiAction.ts	
└─ table/	# 表格相关
└─ useTableSelection.ts	
└─ websocket/	# 实时通信相关 (WebSocket)
└─ useStomp.ts	
└─ useDictSync.ts	
└─ useOnlineCount.ts	
└─ index.ts	# 统一导出
└─ constants/	# 常量定义
└─ index.ts	
└─ directives/	# 自定义指令
└─ permission/	# 权限指令 v-permission
└─ enums/	# 枚举定义
└─ api.ts	# API 相关枚举
└─ settings.ts	# 设置相关枚举
└─ lang/	# 国际化
└─ package/	# 语言包
└─ zh-cn.json	# 中文
└─ en.json	# 英文
└─ index.ts	# 入口配置
└─ utils.ts	# 工具函数
└─ layouts/	# 布局组件
└─ components/	# 布局子组件
└─ index.vue	# 主布局
└─ useLayout.ts	# 布局逻辑
└─ plugins/	# 插件配置
└─ vxe-table.ts	# VXE Table 配置
└─ router/	# 路由配置
└─ guards/	# 路由守卫
└─ index.ts	# 路由实例
└─ store/	# 状态管理
└─ modules/	# Store 模块
└─ user.ts	
└─ app.ts	
└─ permission.ts	
└─ index.ts	# Store 入口
└─ styles/	# 全局样式
└─ variables.scss	# 变量定义
└─ index.scss	# 入口文件
└─ types/	# 类型定义
└─ api/	# API 类型
└─ global.d.ts	# 全局声明
└─ utils/	# 工具函数
└─ request.ts	# HTTP 请求
└─ auth.ts	# 认证工具
└─ storage.ts	# 存储工具
└─ views/	# 页面组件
└─ dashboard/	# 仪表盘
└─ demo/	# 功能演示
└─ system/	# 系统管理
└─ user/	# 用户管理
└─ role/	# 角色管理
└─ menu/	# 菜单管理
└─ login/	# 登录页
└─ App.vue	# 根组件
└─ main.ts	# 入口文件
└─ settings.ts	# 应用配置
└─ tests/	# 测试文件
└─ setup.ts	# 测试配置
└─ types/	# 全局类型声明
└─ .env.development	# 开发环境变量
└─ .env.production	# 生产环境变量
└─ index.html	# HTML 模板
└─ package.json	
└─ tsconfig.json	
└─ uno.config.ts	# UnoCSS 配置
└─ vite.config.ts	# Vite 配置
└─ vitest.config.ts	# Vitest 配置

目录说明

api - 接口层

按业务模块组织 API 接口，单文件模块直接放在 api 根目录。

```
api/
├── system/           # 多文件模块用目录
│   ├── user.ts
│   └── role.ts
├── auth.ts           # 单文件模块直接放根目录
└── file.ts
```

assets - 静态资源

存放需要经过构建处理的静态资源。

```
assets/
├── icons/             # SVG 图标（通过 UnoCSS Icons 加载）
└── images/            # 图片资源（会被压缩优化）
```

1
2
3

components - 公共组件

存放跨页面复用的公共组件，项目已重构为以下组件：

```
components/
├── AiAssistant/       # AI 助手组件
├── Breadcrumb/        # 面包屑导航
├── CommandPalette/    # 命令面板（快捷搜索）
├── CopyButton/        # 复制按钮
├── CURD/              # CURD 快速开发组件
│   ├── PageSearch.vue
│   ├── PageContent.vue
│   ├── PageModal.vue
│   ├── types.ts
│   └── usePage.ts
├── DictSelect/        # 字典选择器
├── DictTag/           # 字典标签
├── ECharts/           # 图表组件
├── Fullscreen/        # 全屏切换
├── Hamburger/        # 菜单折叠按钮
├── IconSelect/        # 图标选择器
├── InputTag/          # 标签输入
├── LangSelect/        # 语言选择
├── NoticeDropdown/    # 通知下拉
├── OperationColumn/   # 操作列
├── Pagination/        # 分页组件
├── SizeSelect/        # 尺寸选择
├── TableSelect/       # 表格选择器
├── TenantSwitcher/    # 租户切换
├── TextScroll/        # 文字滚动
├── ThemeSwitch/       # 主题切换
├── Upload/            # 上传组件
│   ├── FileUpload.vue
│   ├── MultiImageUpload.vue
│   └── SingleImageUpload.vue
└── WangEditor/        # 富文本编辑器
```

命名规范：PascalCase，多词命名

composables - 组合式函数

存放可复用的组合式函数（Composition API）。



```
composables/
├─ ai/
│   └─ useAiAction.ts      # AI 操作助手
├─ table/
│   └─ useTableSelection.ts # 表格选择管理
├─ websocket/
│   ├── useStomp.ts        # STOMP 协议
│   ├── useDictSync.ts     # 字典同步
│   └─ useOnlineCount.ts   # 在线人数
└─ index.ts                # 统一导出
```

命名规范： use 前缀 + camelCase

lang - 国际化

存放多语言配置和语言包。



```
lang/
├─ package/
│   ├── zh-cn.json        # 中文语言包
│   └─ en.json            # 英文语言包
├─ index.ts               # i18n 配置入口
└─ utils.ts               # 工具函数
```

layouts - 布局组件

存放页面布局相关组件。



```
layouts/
├─ components/           # 布局子组件
├─ index.vue             # 主布局入口
└─ useLayout.ts          # 布局逻辑
```

store - 状态管理

使用 Pinia 进行状态管理，按功能模块拆分。



```
store/
├─ modules/
│   ├── user.ts           # 用户状态
│   ├── app.ts            # 应用状态
│   ├── permission.ts     # 权限状态
│   └─ settings.ts        # 设置状态
└─ index.ts               # 统一导出
```

views - 页面组件

按路由/功能模块组织页面组件。



```
views/
├─ dashboard/
│   └─ index.vue
├─ demo/                # 功能演示页面
│   └─ curd/
└─ dictionary.vue
```

```
|
|   |-- icons.vue
|   |-- ...
|-- system/
|   |-- user/
|   |   |-- index.vue
|   |   |-- components/
|   |-- role/
|   |   |-- index.vue
|-- login/
|   |-- index.vue
```

设计原则

1. 单一职责

每个目录只负责一类功能：

- **api/** - 只放 API 接口
- **utils/** - 只放工具函数
- **components/** - 只放公共组件

2. 就近原则

页面私有组件放在页面目录下：

```
● ● ●
views/system/user/
|-- index.vue
|-- components/    # 只有 user 页面使用的组件
|   |-- UserForm.vue
```

3. 扁平优先

避免过深的目录嵌套，单文件模块不建目录：

```
● ● ●
# [x] 好
api/auth.ts

# [x] 不好
api/auth/index.ts    # 只有一个文件，不需要目录
```

4. 统一导出

每个模块目录提供 **index.ts** 统一导出：

```
● ● ●
// composables/index.ts
export * from "../table/useTableSelection";
export * from "../websocket";
```

使用时：

```
● ● ●
import { useTableSelection, useDictSync } from "@composables";
```

模块依赖关系



views (页面)
↓
components (组件) + composables (组合函数)
↓
store (状态) + api (接口)
↓
utils (工具) + types (类型) + constants (常量)

原则：上层可以依赖下层，下层不能依赖上层。

路由和菜单

了解 vue3-element-admin 的路由配置和菜单管理。

项目路由分为两个部分：**静态路由** 和 **动态路由**。

静态路由：通用不需要权限都可以访问的视图。

动态路由：需要通过登录后授权才可以访问的视图。

路由配置

静态路由配置：在 `src/router/index.ts` 中

注册、登录、首页控制台、401、404、个人中心、我的通知、详情页缓存等。



```
import type { App } from "vue";
import { createRouter, createWebHashHistory, type RouteRecordRaw } from "vue-router";

export const Layout = () => import("@/Layouts/index.vue");

// 静态路由
export const constantRoutes: RouteRecordRaw[] = [
  {
    path: "/redirect",
    component: Layout,
    meta: { hidden: true },
    children: [
      {
        path: "/redirect/:path(.*)",
        component: () => import("@/views/redirect/index.vue"),
      },
    ],
  },
  {
    path: "/Login",
    component: () => import("@/views/Login/index.vue"),
    meta: { hidden: true },
  },
  {
    path: "/",
    name: "/",
    component: Layout,
    redirect: "/dashboard",
    children: [
      {
        path: "dashboard",
        component: () => import("@/views/dashboard/index.vue"),
        // 用于 keep-alive 功能, 需要与 SFC 中自动推导或显式声明的组件名称一致
        // 参考文档: https://cn.vuejs.org/guide/built-ins/keep-alive.html#include-exclude
        name: "Dashboard",
        meta: {
          title: "dashboard",
          icon: "homepage",
          affix: true,
          keepAlive: true,
        },
      },
    ],
  },
  {
```

```

    path: "401",
    component: () => import("@/views/error/401.vue"),
    meta: { hidden: true },
  },
  {
    path: "404",
    component: () => import("@/views/error/404.vue"),
    meta: { hidden: true },
  },
  {
    path: "profile",
    name: "Profile",
    component: () => import("@/views/profile/index.vue"),
    meta: { title: "个人中心", icon: "user", hidden: true },
  },
  {
    path: "my-notice",
    name: "MyNotice",
    component: () => import("@/views/profile/notice/index.vue"),
    meta: { title: "我的通知", icon: "user", hidden: true },
  },
  {
    path: "/detail/:id(\\d+)",
    name: "DemoDetail",
    component: () => import("@/views/demo/detail.vue"),
    meta: { title: "详情页缓存", icon: "user", hidden: true, keepAlive: true },
  },
],
},
];

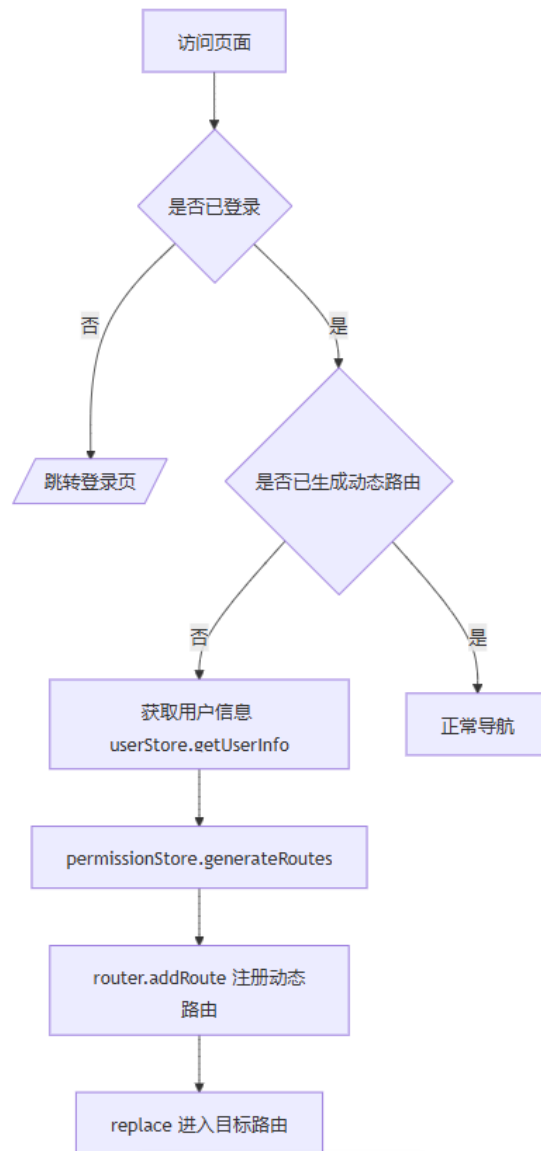
/**
 * 创建路由
 */
const router = createRouter({
  history: createWebHashHistory(),
  routes: constantRoutes,
  // 刷新时，滚动条位置还原
  scrollBehavior: () => ({ left: 0, top: 0 }),
});

// 全局注册 router
export function setupRouter(app: App<Element>) {
  app.use(router);
}

export default router;

```

动态路由配置：在 `src/router/guards/permission.ts` 中，根据用户权限从后端获取，主链路如下：



在store中进行全局路由初始化： `src/store/permission.ts`

```

// src/store/permission.ts
const data = await MenuAPI.getRoutes(); // 获取当前登录人的菜单路由
const dynamicRoutes = transformRoutes(data);

// 加载 静态路由 和 动态路由
routes.value = [...constantRoutes, ...dynamicRoutes];

```

对应的API接口： `src/api/system/menu/index.ts`

```

const MenuAPI = {
  ...
}

```

增加自定义菜单

启用 Mock 服务：修改配置文件 `.env.development`，不启动则使用远程服务端数据

```

# 应用端口
VITE_APP_PORT=3000
# 项目名称
VITE_APP_TITLE=vue3-element-admin
# 代理前缀
VITE_APP_BASE_API=/dev-api

# 接口地址
VITE_APP_API_URL=https://api.youlai.tech      # 线上
# VITE_APP_API_URL=http://localhost:8000      # 本地

# 启用 Mock 服务(true:开启 false:关闭)
VITE MOCK_DEV_SERVER=true                    ##### 把 false 修改为 true

# 多租户开关(true:开启 false:关闭)
VITE_APP_TENANT_ENABLED=false

```

修改菜单的mock数据(本地mock测试数据): 修改文件 `mock\menu.mock.ts`

使用到图标: <https://element-plus.org/zh-CN/component/icon>

```

export default defineMock([
  {
    url: "menus/routes",
    method: ["GET"],
    body: {
      code: "00000",
      data: [
        //-----增加以下代码-----
        {
          path: "/order",
          component: "Layout",
          redirect: "/order/order",
          name: "order",
          meta: {
            title: "订单模块",
            icon: "el-icon-Box",
            hidden: false,
            alwaysShow: false,
            params: null,
          },
        },
        {
          path: "order",
          component: "order/order/index", // 在views目录下创建对应文件 order/order/index.vue
          name: "Order",
          meta: {
            title: "订单管理",
            icon: "el-icon-TakeawayBox",
            hidden: false,
            keepAlive: true,
            alwaysShow: false,
            params: null,
          },
        },
      ],
    },
  },
  {
    path: "track",
    component: "order/track/index", // 在views目录下创建对应文件 order/track/index.vue
    name: "Track",
    meta: {
      title: "订单跟踪",
      icon: "el-icon-PhoneFilled",
      hidden: false,
      keepAlive: true,
      alwaysShow: false,
      params: null,
    },
  },
],
//-----增加以上代码-----
[
  {
    path: "/system",
    component: "Layout",
    redirect: "/system/user",
    name: "system",
  },
]

```

模块视图配置

创建模块视图

创建对应的模块视图文件: `src\views\order\order\index.vue` 对应上面菜单 `component: "order/order/index"`, 属性

```
<template>
  <div class="page-container">订单管理模块页面</div>
</template>

<script setup lang="ts"></script>

<style scoped lang="scss"></style>
```

创建对应的模块视图文件: `src\views\order\track\index.vue` 对应上面菜单 `component: "order/track/index"`, 属性

```
<template>
  <div class="page-container">订单跟踪模块页面</div>
</template>

<script setup lang="ts"></script>

<style scoped lang="scss"></style>
```

启动项目并查看效果:



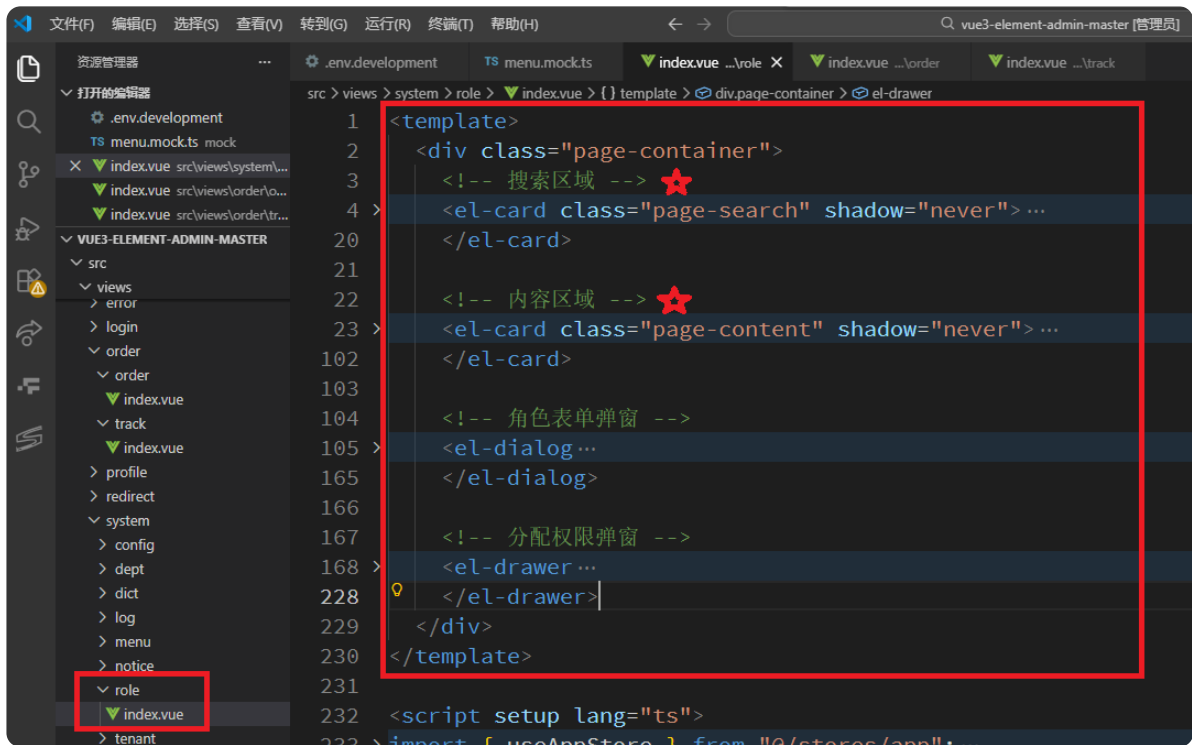
视图布局

Container 容器布局: <https://element-plus.org/zh-CN/component/container>

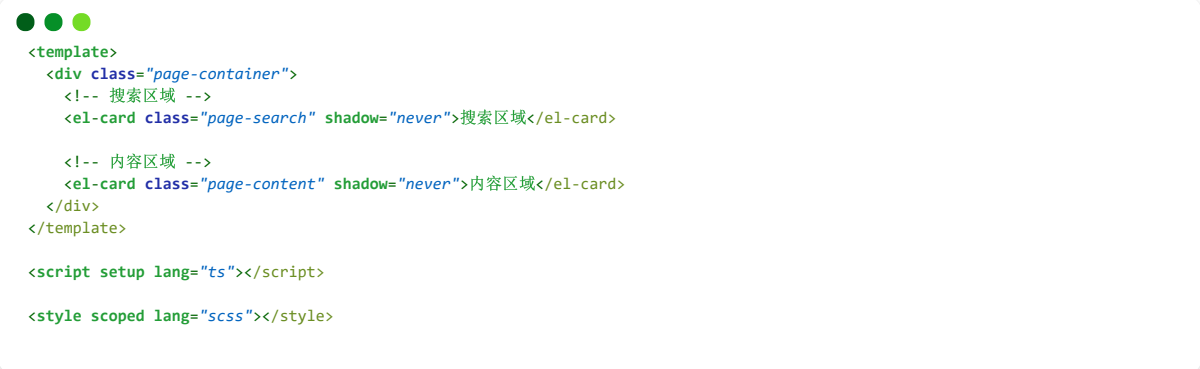
Layout 布局 (24格网格布局): <https://element-plus.org/zh-CN/component/layout>

Card 卡片: <https://element-plus.org/zh-CN/component/card>

参考 角色管理模块 `src\views\system\role\index.vue` 进行 订单管理模块 开发:



修改 `src/views/order/order/index.vue` : 增加搜索区域和内容区域



查看效果:



表格组件 - 显示数据

1. 编写订单模块的mock测试数据

参考 `mock/role.mock.ts` 编写 `mock/order.mock.ts`

业务描述	请求方式	请求url地址	请求参数
------	------	---------	------

业务描述	请求方式	请求url地址	请求参数
查询所有的订单	GET	orders	?pageNum=1&pageSize=10

```

import { defineMock } from "../base";

export default defineMock([
  // 查询所有的订单(分页查询)
  {
    url: "orders",
    method: ["GET"],
    body: {
      code: "00000",
      data: {
        list: [
          {
            id: "1",
            code: "0-0000001",
            name: "App研发订单",
            createTime: "2021-03-25 12:39:54",
            updateTime: null,
          },
          {
            id: "2",
            code: "0-0000002",
            name: "App研发订单2",
            createTime: "2021-03-25 12:39:54",
            updateTime: null,
          },
          {
            id: "3",
            code: "0-0000003",
            name: "App研发订单3",
            createTime: "2021-03-25 12:39:54",
            updateTime: null,
          },
        ],
        total: 3,
      },
      msg: "一切ok",
    },
  },
]);

```

2.编写订单模块的API接口的类型定义

编写订单模块的数据类型文件: `src/api/order/order/types.ts`

```

/**
 * Order 订单模块 类型定义
 */
import type { BaseQueryParams } from "@api/common";

/** 角色分页查询参数 */
export interface OrderQueryParams extends BaseQueryParams {
  /** 搜索关键字 */
  keywords?: string;
}

// 常用的数据类型 string、number、Date 等 js数据类型
/** 订单分页对象 */
export interface OrderItem {
  /** 订单ID */
  id?: string;
  /** 订单编码 */
  code?: string;
  /** 订单名称 */
  name?: string;
  /** 创建时间 */
  createTime?: Date;
  /** 修改时间 */
  updateTime?: Date;
}

```

3.编写订单模块的API接口

编写订单模块的API接口文件: `src/api/order/order/index.ts`

```

import request from "@utils/request"; // 封装Axios 简化网络请求接口编写

```

```
import type { OrderQueryParams, OrderItem } from "../types";

const ORDER_BASE_URL = "/api/v1/orders";

const OrderAPI = {
  /** 获取角色分页数据 */
  getPage(queryParams?: OrderQueryParams) {
    return request<any, PageResult<OrderItem>>({
      url: `${ORDER_BASE_URL}`,
      method: "get",
      params: queryParams,
    });
  },
};

export default OrderAPI;

// 重导出类型
export * from "../types";
```

Axios 是一个基于 promise 的网络请求库，可以用于浏览器和 node.js

Axios 使用简单,包尺寸小且提供了易于扩展的接口。

<https://www.axios-http.cn/>

4.绑定API接口数据到Table组件

Table 表格: <https://element-plus.org/zh-CN/component/table>

参考 `src\views\system\role\index.vue` 编写 `src\views\order\order\index.vue`

```
<template>
  <div class="page-container">
    <!-- 搜索区域 -->
    <el-card class="page-search" shadow="never">搜索区域</el-card>

    <!-- 内容区域 -->
    <el-card class="page-content" shadow="never">
      <!-- 添加一个数据表格组件 ref="dataTableRef" @selection-change="handleSelectionChange"

      /** 订单ID */
      id?: string;
      /** 订单编码 */
      code?: string;
      /** 订单名称 */
      name?: string;
      /** 创建时间 */
      createTime?: Date;
      /** 修改时间 */
      updateTime?: Date;
      -->
      <el-table v-loading="loading" :data="orderList" highlight-current-row border>
        <el-table-column type="selection" width="55" align="center" />
        <el-table-column label="ID" prop="id" width="100" />
        <el-table-column label="订单编码" prop="code" width="150" />
        <el-table-column label="订单名称" prop="name" min-width="180" />
        <el-table-column label="创建时间" prop="createTime" width="160" />
        <el-table-column label="修改时间" prop="updateTime" width="160" />
      </el-table>
    </el-card>
  </div>
</template>

<script setup lang="ts">
  // 1. 导入订单模块的API
  import OrderAPI from "@api/order/order";
  // 2. 导入订单模块的API的数据封装对象
  import type { OrderItem, OrderQueryParams } from "@api/order/order"; // OrderForm

  // 3. 页面数据加载特效变量
  const loading = ref(false);
  // 4. 查询参数对象 (分页参数)
  const queryParams = reactive<OrderQueryParams>({
    pageNum: 1,
    pageSize: 10,
  });

  // 5. 订单表格数据
  const orderList = ref<OrderItem[]>();

  // 6. 页面总记录数变量
  const total = ref(0);

  /**
```

```
* 7. 加载订单列表数据
*/
async function fetchList(): Promise<void> {
  // 启动加载数据动画特效
  loading.value = true;
  try {
    // 调用 OrderAPI 接口 获取分页查询结果集合数据
    const data = await OrderAPI.getPage(queryParams);
    // 获取结果集data中数据列表list并赋值给（响应式）变量orderList
    orderList.value = data.list;
    // 获取结果集data中总记录数total并赋值给（响应式）变量total
    total.value = data.total ?? 0;
  } finally {
    // 关闭加载数据动画特效
    loading.value = false;
  }
}

// 8. 查询（重置页码后获取数据）
function handleQuery(): void {
  // 重置查询参数 页面编号为 1 第一页
  queryParams.pageNum = 1;
  // 调用fetchList获取分页查询结果集
  fetchList();
}

// 9. 视图组件挂载完成后 查询表格组件的第一页数据
onMounted(() => {
  handleQuery();
});
</script>

<style scoped lang="scss"></style>
```

表格组件 - 删除数据

1.编写订单模块的mock测试数据

参考 `mock\role.mock.ts` 编写 `mock\order.mock.ts`

业务描述	请求方式	请求uri地址	请求参数
删除订单	DELETE	orders/:id	<code>params.id</code> 赋值给请求地址中的 <code>:id</code> 例如: orders/1 或者 orders/1,2
查询所有的订单	GET	orders	?pageNum=1&pageSize=10



```
import { defineMock } from "../base";

export default defineMock([
  // 查询所有的订单(分页查询)
  {
    url: "orders",
    method: ["GET"],
    body: {
      code: "00000",
      data: {
        list: [
          {
            id: "1",
            code: "0-0000001",
            name: "App研发订单",
            createTime: "2021-03-25 12:39:54",
            updateTime: "2021-03-25 12:39:54",
          },
          {
            id: "2",
            code: "0-0000002",
            name: "App研发订单2",
            createTime: "2021-03-25 12:39:54",
            updateTime: "2021-03-25 12:39:54",
          },
          {
            id: "3",
            code: "0-0000003",
            name: "App研发订单3",
            createTime: "2021-03-25 12:39:54",
            updateTime: "2021-03-25 12:39:54",
          },
        ],
        total: 3,
      },
    },
  },
]);
```

```

    },
    msg: "一切ok",
  },
},
// 删除订单 params => :id
{
  url: "orders/:id",
  method: ["DELETE"],
  body({ params }) {
    return {
      code: "00000",
      data: null,
      msg: "删除订单" + params.id + "成功",
    };
  },
},
},
]);

```

2.编写订单模块的API接口

参考: `src\api\system\role\index.ts` 编写订单模块的API接口文件: `src\api\order\order\index.ts`

```

import request from "@utils/request";
import type { OrderQueryParams, OrderItem } from "./types";

const ORDER_BASE_URL = "/api/v1/orders";

const OrderAPI = {
  /** 获取角色分页数据 */
  getPage(queryParams?: OrderQueryParams) {
    return request<any, PageResult<OrderItem>>({
      url: `${ORDER_BASE_URL}`,
      method: "get",
      params: queryParams,
    });
  },
  /** 批量删除订单 (根据id进行删除), 多个id以英文逗号(,)分割 */
  deleteByIds(ids: string) {
    return request({ url: `${ORDER_BASE_URL}/${ids}`, method: "delete" });
  },
};

export default OrderAPI;

// 重导出类型
export * from "./types";

```

3.Table组件增加删除功能

Table 表格: <https://element-plus.org/zh-CN/component/table>

增加 **删除行** 和 **批量删除** 按钮和事件

参考 `src\views\system\role\index.vue` 编写 `src\views\order\order\index.vue`

```

<template>
  <div class="page-container">
    <!-- 搜索区域 -->
    <el-card class="page-search" shadow="never">搜索区域</el-card>

    <!-- 内容区域 -->
    <el-card class="page-content" shadow="never">
      <div class="page-toolbar">
        <div class="page-toolbar__left">
          <!-- 新增事件 @click="handleCreateClick()" -->
          <el-button type="success" icon="plus">新增</el-button>
          <!-- 批量删除事件 -->
          <el-button
            type="danger"
            :disabled="ids.length === 0"
            icon="delete"
            @click="handleBatchDelete()"
          >
            删除
          </el-button>
        </div>
      </div>
      <!-- 添加一个数据表格组件 ref="dataTableRef" -->
      <el-table

```



```

v-loading="Loading"
:data="orderList"
highlight-current-row
border
@selection-change="handleSelectionChange"
>
<el-table-column type="selection" width="55" align="center" />
<el-table-column label="ID" prop="id" width="100" />
<el-table-column label="订单编码" prop="code" width="150" />
<el-table-column label="订单名称" prop="name" min-width="180" />
<el-table-column label="创建时间" prop="createTime" width="160" />
<el-table-column label="修改时间" prop="updateTime" width="160" />

<el-table-column fixed="right" label="操作" width="220">
  <template #default="scope">
    <!-- 编辑当前行事件 @click="handleEditClick(scope.row.id)" -->
    <el-button type="primary" size="small" link icon="edit">编辑</el-button>
    <!-- 删除当前行事件 -->
    <el-button
      type="danger"
      size="small"
      link
      icon="delete"
      @click="handleDelete(scope.row.id)"
    >
      删除
    </el-button>
  </template>
</el-table-column>
</el-table>
</el-card>
</div>
</template>

<script setup lang="ts">
// 1. 导入订单模块的API
import OrderAPI from "@api/order/order";
// 2. 导入订单模块的API的数据封装对象
import type { OrderItem, OrderQueryParams } from "@api/order/order"; // OrderForm

// 3. 页面数据加载特效变量
const loading = ref(false);
// 4. 查询参数对象（分页参数）
const queryParams = reactive<OrderQueryParams>({
  pageNum: 1,
  pageSize: 10,
});

// 5. 订单表格数据
const orderList = ref<OrderItem[]>();

// 6. 页面总记录数变量
const total = ref(0);

/**
 * 7. 加载订单列表数据
 */
async function fetchList(): Promise<void> {
  // 启动加载数据动画特效
  loading.value = true;
  try {
    // 调用 OrderAPI 接口 获取分页查询结果集合数据
    const data = await OrderAPI.getPage(queryParams);
    // 获取结果集data中数据列表list并赋值给（响应式）变量orderList
    orderList.value = data.list;
    // 获取结果集data中总记录数total并赋值给（响应式）变量total
    total.value = data.total ?? 0;
  } finally {
    // 关闭加载数据动画特效
    loading.value = false;
  }
}

// 8. 查询（重置页码后获取数据）
function handleQuery(): void {
  // 重置查询参数 页面编号为 1 第一页
  queryParams.pageNum = 1;
  // 调用fetchList获取分页查询结果集
  fetchList();
}

/**
 * 重置查询条件
 */
function resetQuery(): void {
  // 查询功能后面再实现 重置查询字段的文本框
  // queryFormRef.value?.resetFields();
}

/**
 * 重置查询条件并重新查询
 */
function handleResetQuery(): void {
  resetQuery();
  handleQuery();
}

```

```
}
// 批量删除的ids数组
const ids = ref<number[]>([]);

// 复选框选中事件（批量删除）
function handleSelectionChange(selection: any): void {
  ids.value = selection.map((item: any) => item.id);
}

// 删除订单
function handleDelete(id?: number): void {
  // 传参数id 就是删除一条记录，不传参数id 自动获取 ids进行批量删除 并使用英文逗号链接成新的字符串
  const delIds = id ? String(id) : ids.value.join(",");
  if (!delIds) {
    ElMessage.warning("请勾选删除项");
    return;
  }

  ElMessageBox.confirm("确认删除已选中的数据项?", "警告", {
    confirmButtonText: "确定",
    cancelButtonText: "取消",
    type: "warning",
  }).then(
    () => {
      loading.value = true;
      // console.log("delIds:" + delIds);
      // 调用OrderAPI的删除方法
      OrderAPI.deleteByIds(delIds)
        .then(() => {
          ElMessage.success("删除成功");
          handleResetQuery(); // 重置查询
        })
        .finally(() => (loading.value = false));
    },
    () => {
      ElMessage.info("已取消删除");
    }
  );
}

/**
 * 批量删除按钮点击事件
 */
function handleBatchDelete(): void {
  handleDelete();
}

// 9. 视图组件挂载完成后 查询表格组件的第一页数据
onMounted(() => {
  handleQuery();
});
</script>

<style scoped lang="scss"></style>
```

表格组件 - 编辑数据

1.编写订单模块的mock测试数据

参考 `mock\role.mock.ts` 编写 `mock\order.mock.ts`

业务描述	请求方式	请求url地址	请求参数
修改订单	PUT	orders	data: OrderForm
根据id返回订单数据（编辑表单）	GET	orders/:id	<code>params.id</code> 赋值给请求地址中的 <code>:id</code>
删除订单	DELETE	orders/:id	<code>params.id</code> 赋值给请求地址中的 <code>:id</code>
查询所有的订单	GET	orders	?pageNum=1&pageSize=10



```
import { defineMock } from "../base";

export default defineMock([
  // 查询所有的订单(分页查询)
  {
    url: "orders",
    method: ["GET"],
    body: {
      code: "00000",
      data: {
```

```

list: [
  {
    id: "1",
    code: "0-0000001",
    name: "App 研发订单",
    createTime: "2021-03-25 12:39:54",
    updateTime: "2021-03-25 12:39:54",
  },
  {
    id: "2",
    code: "0-0000002",
    name: "App 研发订单2",
    createTime: "2021-03-25 12:39:54",
    updateTime: "2021-03-25 12:39:54",
  },
  {
    id: "3",
    code: "0-0000003",
    name: "App 研发订单3",
    createTime: "2021-03-25 12:39:54",
    updateTime: "2021-03-25 12:39:54",
  },
],
total: 3,
},
msg: "一切ok",
},
},
// 删除订单 params => :id
{
  url: "orders/:id",
  method: ["DELETE"],
  body({ params }) {
    return {
      code: "00000",
      data: null,
      msg: "删除订单" + params.id + "成功",
    };
  },
},
// 根据id获取 1条 订单数据
{
  url: "orders/:id",
  method: ["GET"],
  body: ({ params }) => {
    return {
      code: "00000",
      data: orderMap[params.id],
      msg: "一切ok",
    };
  },
},
// 修改订单 body 为提交表单的荷载数据
{
  url: "orders",
  method: ["PUT"],
  body({ body }) {
    return {
      code: "00000",
      data: null,
      msg: "修改订单" + body.code + "成功",
    };
  },
},
});

// 订单映射表数据 根据id返回数据
const orderMap: Record<string, any> = {
  1: {
    id: "1",
    code: "0-0000001",
    name: "App 研发订单",
    createTime: "2021-03-25 12:39:54",
    updateTime: "2021-03-25 12:39:54",
  },
  2: {
    id: "2",
    code: "0-0000002",
    name: "App 研发订单2",
    createTime: "2021-03-25 12:39:54",
    updateTime: "2021-03-25 12:39:54",
  },
  3: {
    id: "3",
    code: "0-0000003",
    name: "App 研发订单3",
    createTime: "2021-03-25 12:39:54",
    updateTime: "2021-03-25 12:39:54",
  },
};

```

2.编写订单模块的API接口的类型定义

编写订单模块的数据类型文件: `src\api\order\order\types.ts`

```
/**
 * Order 订单模块 类型定义 Typescript
 */

import type { BaseQueryParams } from "@api/common";

/** 角色分页查询参数 */
export interface OrderQueryParams extends BaseQueryParams {
  /** 搜索关键字 */
  keywords?: string;
}

// 常用的数据类型 string、number、Date 等 js数据类型
/** 订单分页对象 */
export interface OrderItem {
  /** 订单ID */
  id?: string;
  /** 订单编码 */
  code?: string;
  /** 订单名称 */
  name?: string;
  /** 创建时间 */
  createTime?: Date;
  /** 修改时间 */
  updateTime?: Date;
}

/** 订单模块的表单对象 */
export interface OrderForm {
  /** 订单ID */
  id?: string;
  /** 订单编码 */
  code?: string;
  /** 订单名称 */
  name?: string;
  /** 创建时间 */
  createTime?: Date;
  /** 修改时间 */
  updateTime?: Date;
}
```

3.编写订单模块的API接口

参考: `src\api\system\role\index.ts` 编写订单模块的API接口文件: `src\api\order\order\index.ts`

```
import request from "@utils/request";
import type { OrderQueryParams, OrderItem, OrderForm } from "../types";

const ORDER_BASE_URL = "/api/v1/orders";

const OrderAPI = {
  /** 获取角色分页数据 */
  getPage(queryParams?: OrderQueryParams) {
    return request<any, PageResult<OrderItem>>({
      url: `${ORDER_BASE_URL}`,
      method: "get",
      params: queryParams,
    });
  },
  /** 批量删除订单 (根据id进行删除), 多个id以英文逗号(,)分割 */
  deleteByIds(ids: string) {
    return request({ url: `${ORDER_BASE_URL}/${ids}`, method: "delete" });
  },
  /** 根据id获取订单数据 绑定到表单 */
  getFormData(id: string) {
    return request<any, OrderForm>({ url: `${ORDER_BASE_URL}/${id}`, method: "get" });
  },
  /** 更新订单明细 */
  update(data: OrderForm) {
    return request({ url: `${ORDER_BASE_URL}`, method: "put", data });
  },
};

export default OrderAPI;

// 重导出类型
export * from "../types";
```

4. Table组件增加编辑功能

Table 表格: <https://element-plus.org/zh-CN/component/table>

增加 **编辑** 按钮和事件

参考 `src\views\system\role\index.vue` 编写 `src\views\order\order\index.vue`

```
<template>
  <div class="page-container">
    <el-card class="page-search" shadow="never">搜索区域</el-card>
    <el-card class="page-content" shadow="never">
      <div class="page-toolbar">
        <div class="page-toolbar__left">
          <!-- <el-button type="success" icon="plus" @click="handleCreateClick()">新增</el-button> -->
          <el-button
            type="danger"
            :disabled="ids.length === 0"
            icon="delete"
            @click="handleBatchDelete()"
          >
            批量删除
          </el-button>
        </div>
      </div>
      <!--
      ref="dataTableRef"
      -->
      <el-table
        v-loading="Loading"
        :data="orderList"
        highlight-current-row
        border
        @selection-change="handleSelectionChange"
      >
        <!-- 复选框 -->
        <el-table-column type="selection" width="55" align="center" />
        <!-- 订单业务字段 -->
        <el-table-column label="订单名称" prop="name" min-width="100" />
        <el-table-column label="订单编码" prop="code" width="150" />
        <!-- 系统字段 -->
        <el-table-column label="创建时间" prop="createTime" width="160" />
        <el-table-column label="修改时间" prop="updateTime" width="160" />

        <el-table-column fixed="right" label="操作" width="220">
          <template #default="scope">
            <el-button
              type="primary"
              size="small"
              link
              icon="edit"
              @click="handleEditClick(scope.row.id)"
            >
              编辑
            </el-button>
            <el-button
              type="danger"
              size="small"
              link
              icon="delete"
              @click="handleDelete(scope.row.id)"
            >
              删除
            </el-button>
          </template>
        </el-table-column>
      </el-table>
      <!-- 分页组件 total、page、limit、@pagination分页事件 - 调用分页查询方法-->
      <pagination
        v-if="total > 0"
        v-model:total="total"
        v-model:page="queryParams.pageNum"
        v-model:limit="queryParams.pageSize"
        @pagination="fetchList"
      />
    </el-card>

    <!-- 订单表单弹窗 -->
    <el-dialog
      v-model="dialogState.visible"
      :title="dialogState.title"
      width="600px"
      @close="closeDialog"
    >
      <el-form ref="orderFormRef" :model="formData" :rules="rules" label-width="100px">
        <el-form-item label="订单名称" prop="name">
          <el-input v-model="formData.name" placeholder="请输入订单名称" />
        </el-form-item>

        <el-form-item label="订单编码" prop="code">
```

```

        <el-input v-model="formData.code" placeholder="请输入订单编码" />
      </el-form-item>
    </el-form>
    <template #footer>
      <div class="dialog-footer">
        <el-button type="primary" @click="handleSubmit">确定</el-button>
        <el-button @click="closeDialog">取消</el-button>
      </div>
    </template>
  </el-dialog>
</div>
</template>
<script setup lang="ts">
// 引入 订单管理模块 API 对象
import OrderAPI from "@api/order/order";
// 引入 订单管理模块 数据封装对象
import { OrderItem, OrderQueryParams, OrderForm } from "@api/order/order";

// 表格组件加载数据动态特效开关
const loading = ref(false);
// 总记录数
const total = ref(0);
// 分页查询对象
const queryParams = reactive<OrderQueryParams>({
  pageNum: 1,
  pageSize: 10,
});

// 订单表格数据列表
const orderList = ref<OrderItem[]>();
/**
 * 加载订单列表数据
 */
async function fetchList(): Promise<void> {
  // 开启加载数据动画特效
  loading.value = true;
  try {
    // 调用 OrderAPI 获取 分页数据
    const data = await OrderAPI.getPage(queryParams);
    // 从结果集 data 中获取 结果集 list 进行赋值
    orderList.value = data.list;
    // 从结果集 data 中获取 总记录数 total 进行赋值
    total.value = data.total ?? 0;
  } finally {
    // 关闭加载数据动画特效
    loading.value = false;
  }
}

// 查询（重置页码后获取数据）
function handleQuery(): void {
  queryParams.pageNum = 1;
  fetchList();
}

/**
 * 重置查询条件
 */
function resetQuery(): void {
  // queryFormRef.value?.resetFields();
}

/**
 * 重置查询条件并重新查询
 */
function handleResetQuery(): void {
  resetQuery();
  handleQuery();
}

// 删除一行记录 和 批量删除 功能
const ids = ref<number[]>([]); // 用于保存表格 复选框组件 的选中记录的ids 数组

// 行复选框选中
function handleSelectionChange(selection: any): void {
  ids.value = selection.map((item: any) => item.id);
}

// 删除订单
function handleDelete(tempId?: number): void {
  // tempId 有传值 删除1条记录 没有传值 批量删除ids
  const tempIds = tempId ? String(tempId) : ids.value.join(",");
  if (!tempIds) {
    ElMessage.warning("请勾选删除项");
    return;
  }

  ElMessageBox.confirm("确认删除已选中的数据项?", "警告", {
    confirmButtonText: "确定",
    cancelButtonText: "取消",
    type: "warning",
  }).then(
    () => {
      loading.value = true;
      OrderAPI.deleteByIds(tempIds)
    }
  );
}

```

```

        .then(() => {
            ElMessage.success("删除成功");
            handleResetQuery();
        })
        .finally(() => (loading.value = false));
    },
    () => {
        ElMessage.info("已取消删除");
    }
    );
}

/**
 * 批量删除按钮点击事件
 */
function handleBatchDelete(): void {
    handleDelete();
}

// 编辑订单数据功能:
const orderFormRef = ref();

// 弹窗
const dialogState = reactive({
    title: "",
    visible: false, // 弹出窗口 false : 默认为隐藏
});

// 订单的表单对象
const formData = reactive<OrderForm>({});
// 订单表单的校验规则
const rules = reactive({
    name: [{ required: true, message: "请输入角色名称", trigger: "blur" }],
    code: [{ required: true, message: "请输入角色编码", trigger: "blur" }],
});

/**
 * 打开表单弹窗
 */
function openDialog(): void {
    dialogState.visible = true;
}

/**
 * 关闭表单弹窗
 */
function closeDialog(): void {
    dialogState.visible = false;
    resetForm();
}

/**
 * 重置表单数据和验证状态
 */
function resetForm(): void {
    // 1. 重置orderFormRef 表单所有字段
    orderFormRef.value?.resetFields();
    // 2. 重置orderFormRef 表单所有校验
    orderFormRef.value?.clearValidate();

    // 3. 设置 清空业务字段
    formData.id = undefined;
    formData.name = undefined;
    formData.code = undefined;
    formData.createTime = undefined;
    formData.updateTime = undefined;
    // 4. 设置[下拉列表\复选框\单选按钮组] 的默认值
}

/**
 * 编辑按钮点击事件
 * @param roleId 角色ID
 */
async function handleEditClick(id: string): Promise<void> {
    // 1. 设置 弹出窗口 的标题
    dialogState.title = "修改订单";
    // 2. 根据当记录行id 查询要修改的订单数据
    const data = await OrderAPI.getFormData(id);
    // 3. 复制 修改的行数据 到 表单对象上
    Object.assign(formData, data);
    // 4. 打开 弹出窗口
    openDialog();
}

// 提交角色表单
async function handleSubmit(): Promise<void> {
    // 1. 判断表单所有的值是否通过校验
    const valid = await orderFormRef.value?.validate().then(
        () => true,
        () => false
    );
    // 2. 如果没有通过校验 则直接退出提交事件
    if (!valid) return;

    // 3. 如果通过校验 则运行下面代码

```

```
// 3.1 解构 formData 的数据 赋值到 submitData 中
const submitData = { ...formData };
loading.value = true;
try {
  const id = formData.id;
  if (id) {
    await OrderAPI.update(submitData);
    ElMessage.success("修改成功");
  } else {
    // await OrderAPI.create(submitData);
    // ElMessage.success("新增成功");
  }
  closeDialog();
  handleResetQuery();
} finally {
  loading.value = false;
}
}

// 组件挂载后 重置查询条件为第一页 并且 加载订单列表数据
onMounted(() => {
  handleQuery();
});
</script>

<style lang="sass"></style>
```

表格组件 - 新增数据

1.编写订单模块的mock测试数据

参考 `mock\role.mock.ts` 编写 `mock\order.mock.ts`

业务描述	请求方式	请求url地址	请求参数
新增订单	POST	orders	data: OrderForm (id为null)
修改订单	PUT	orders	data: OrderForm (id不为null)
根据id返回订单数据 (编辑表单)	GET	orders/:id	<code>params.id</code> 赋值给请求地址中的 <code>:id</code>
删除订单	DELETE	orders/:id	<code>params.id</code> 赋值给请求地址中的 <code>:id</code>
查询所有的订单	GET	orders	?pageNum=1&pageSize=10



2.编写订单模块的API接口的类型定义

编写订单模块的数据类型文件: `src\api\order\order\types.ts`



```
// 1. 引入 defineMock 定义Mock函数
import { defineMock } from "../base";
// 2. 导出 自定义Mock
export default defineMock([
  // 新增订单
  {
    url: "orders",
    method: ["POST"],
    body({ body }) {
      return {
        code: "00000",
        data: null,
        msg: "新增订单" + body.name + "成功",
      };
    },
  },
],
);

// 获取订单数据 - 填充到编辑表单
{
  url: "orders/:id",
```



```

        method: ["GET"],
        body: ({ params }) => {
            return {
                code: "00000",
                data: orderMap[params.id],
                msg: "一切ok",
            };
        },
    },
    // 修改角色
    {
        url: "orders",
        method: ["PUT"],
        body({ body }) {
            return {
                code: "00000",
                data: null,
                msg: "修改订单" + body.id + "成功",
            };
        },
    },
    // 删除订单
    {
        url: "orders/:id",
        method: ["DELETE"],
        body({ params }) {
            return {
                code: "00000",
                data: null,
                msg: "删除订单" + params.id + "成功",
            };
        },
    },
    // 订单管理模块 分页查询
    {
        url: "orders",
        method: ["GET"],
        body: {
            code: "00000",
            data: {
                list: [
                    {
                        id: "1",
                        name: "中国航空飞机订单",
                        code: "C000001",
                        createTime: "2021-03-25 12:39:54",
                        updateTime: "2021-03-25 12:39:54",
                    },
                    {
                        id: "2",
                        name: "中国航空飞机订单2",
                        code: "C000002",
                        createTime: "2021-03-25 12:39:54",
                        updateTime: "2021-03-25 12:39:54",
                    },
                    {
                        id: "3",
                        name: "中国航空飞机订单3",
                        code: "C000003",
                        createTime: "2021-03-25 12:39:54",
                        updateTime: "2021-03-25 12:39:54",
                    },
                ],
                total: 3,
            },
            msg: "一切ok",
        },
    },
    ]);

const orderMap: Record<string, any> = {
    1: {
        id: "1",
        name: "中国航空飞机订单",
        code: "C000001",
        createTime: "2021-03-25 12:39:54",
        updateTime: "2021-03-25 12:39:54",
    },
    2: {
        id: "2",
        name: "中国航空飞机订单2",
        code: "C000002",
        createTime: "2021-03-25 12:39:54",
        updateTime: "2021-03-25 12:39:54",
    },
    3: {
        id: "3",
        name: "中国航空飞机订单3",
        code: "C000003",
        createTime: "2021-03-25 12:39:54",
        updateTime: "2021-03-25 12:39:54",
    },
};

```

3.编写订单模块的API接口

参考: `src\api\system\role\index.ts` 编写订单模块的API接口文件: `src\api\order\order\index.ts`

```
// 1. 引入 axios 封装的 request 工具方法
import request from "@utils/request";
// 2. 引入 Order 订单模块 定义 的数据封装类
import { OrderQueryParams, OrderItem, OrderForm } from "../types";

// 3. 声明 订单管理模块 请求地址
const ORDER_BASE_URL = "/api/v1/orders";

// 3. 声明 订单管理模块 API 对象
const OrderAPI = {
  /** 新增订单 */
  create(data: OrderForm) {
    return request({ url: `${ORDER_BASE_URL}`, method: "post", data });
  },
  /** 获取订单数据 - 填充编辑表单 */
  getFormData(id: string) {
    return request<any, OrderForm>({ url: `${ORDER_BASE_URL}/${id}`, method: "get" });
  },
  /** 更新订单 */
  update(data: OrderForm) {
    return request({ url: `${ORDER_BASE_URL}`, method: "put", data });
  },
  /** 批量删除订单, 多个id以英文逗号(,)分割 */
  deleteByIds(ids: string) {
    return request({ url: `${ORDER_BASE_URL}/${ids}`, method: "delete" });
  },
  /** 获取订单分页数据 */
  getPage(queryParams?: OrderQueryParams) {
    return request<any, PageResult<OrderItem>>({
      url: `${ORDER_BASE_URL}`,
      method: "get",
      params: queryParams,
    });
  },
};

// 4. 导出 订单管理模块 API 对象 OrderAPI
export default OrderAPI;

// 重导出类型
export * from "../types";
```

4.Table组件增加新增功能

Table 表格: <https://element-plus.org/zh-CN/component/table>

增加 **新增** 按钮和事件

参考 `src\views\system\role\index.vue` 编写 `src\views\order\order\index.vue`

```
<template>
  <div class="page-container">
    <el-card class="page-search" shadow="never">搜索区域</el-card>
    <el-card class="page-content" shadow="never">
      <div class="page-toolbar">
        <div class="page-toolbar__left">
          <el-button type="success" icon="plus" @click="handleCreateClick()">新增</el-button>
          <el-button
            type="danger"
            :disabled="ids.length === 0"
            icon="delete"
            @click="handleBatchDelete()"
          >
            批量删除
          </el-button>
        </div>
      </div>
      <!--
      ref="dataTableRef"
      -->
      <el-table
        v-loading="Loading"
        :data="orderList"
        highlight-current-row
        border
        @selection-change="handleSelectionChange"
```

```

>
<!-- 复选框 -->
<el-table-column type="selection" width="55" align="center" />
<!-- 订单业务字段 -->
<el-table-column label="订单名称" prop="name" min-width="100" />
<el-table-column label="订单编码" prop="code" width="150" />
<!-- 系统字段 -->
<el-table-column label="创建时间" prop="createTime" width="160" />
<el-table-column label="修改时间" prop="updateTime" width="160" />

<el-table-column fixed="right" label="操作" width="220">
  <template #default="scope">
    <el-button
      type="primary"
      size="small"
      link
      icon="edit"
      @click="handleEditClick(scope.row.id)"
    >
      编辑
    </el-button>
    <el-button
      type="danger"
      size="small"
      link
      icon="delete"
      @click="handleDelete(scope.row.id)"
    >
      删除
    </el-button>
  </template>
</el-table-column>
</el-table>
<!-- 分页组件 total、page、Limit、@pagination 分页事件 - 调用分页查询方法-->
<pagination
  v-if="total > 0"
  v-model:total="total"
  v-model:page="queryParams.pageNum"
  v-model:limit="queryParams.pageSize"
  @pagination="fetchList"
/>
</el-card>

<!-- 订单表单弹窗 -->
<el-dialog
  v-model="dialogState.visible"
  :title="dialogState.title"
  width="600px"
  @close="closeDialog"
>
  <el-form ref="orderFormRef" :model="formData" :rules="rules" label-width="100px">
    <el-form-item label="订单名称" prop="name">
      <el-input v-model="formData.name" placeholder="请输入订单名称" />
    </el-form-item>

    <el-form-item label="订单编码" prop="code">
      <el-input v-model="formData.code" placeholder="请输入订单编码" />
    </el-form-item>
  </el-form>
  <template #footer>
    <div class="dialog-footer">
      <el-button type="primary" @click="handleSubmit">确定</el-button>
      <el-button @click="closeDialog">取消</el-button>
    </div>
  </template>
</el-dialog>
</div>
</template>
<script setup lang="ts">
// 引入 订单管理模块 API 对象
import OrderAPI from "@/api/order/order";
// 引入 订单管理模块 数据封装对象
import { OrderItem, OrderQueryParams, OrderForm } from "@/api/order/order";

// 表格组件加载数据动态特效开关
const loading = ref(false);
// 总记录数
const total = ref(0);
// 分页查询对象
const queryParams = reactive<OrderQueryParams>({
  pageNum: 1,
  pageSize: 10,
});

// 订单表格数据列表
const orderList = ref<OrderItem[]>();
/**
 * 加载订单列表数据
 */
async function fetchList(): Promise<void> {
  // 开启加载数据动画特效
  loading.value = true;
  try {
    // 调用 OrderAPI 获取 分页数据
    const data = await OrderAPI.getPage(queryParams);
  }
}

```

```

    // 从结果集 data 中获取 结果集 list 进行赋值
    orderList.value = data.list;
    // 从结果集 data 中获取 总记录数 total 进行赋值
    total.value = data.total ?? 0;
  } finally {
    // 关闭加载数据动画特效
    loading.value = false;
  }
}

// 查询（重置页码后获取数据）
function handleQuery(): void {
  queryParams.pageNum = 1;
  fetchList();
}

/**
 * 重置查询条件
 */
function resetQuery(): void {
  // queryFormRef.value?.resetFields();
}

/**
 * 重置查询条件并重新查询
 */
function handleResetQuery(): void {
  resetQuery();
  handleQuery();
}

// 删除一行记录 和 批量删除 功能
const ids = ref<number[]>([]); // 用于保存表格 复选框组件 的选中记录的ids 数组

// 行复选框选中
function handleSelectionChange(selection: any): void {
  ids.value = selection.map((item: any) => item.id);
}

// 删除订单
function handleDelete(tempId?: number): void {
  // tempId 有传值 删除1条记录 没有传值 批量删除ids
  const tempIds = tempId ? String(tempId) : ids.value.join(",");
  if (!tempIds) {
    ElMessage.warning("请勾选删除项");
    return;
  }

  ElMessageBox.confirm("确认删除已选中的数据项?", "警告", {
    confirmButtonText: "确定",
    cancelButtonText: "取消",
    type: "warning",
  }).then(
    () => {
      loading.value = true;
      OrderAPI.deleteByIds(tempIds)
        .then(() => {
          ElMessage.success("删除成功");
          handleResetQuery();
        })
        .finally(() => (loading.value = false));
    },
    () => {
      ElMessage.info("已取消删除");
    }
  );
}

/**
 * 批量删除按钮点击事件
 */
function handleBatchDelete(): void {
  handleDelete();
}

// 编辑订单数据功能:
const orderFormRef = ref();

// 弹窗
const dialogState = reactive({
  title: "",
  visible: false, // 弹出窗口 false : 默认为隐藏
});

// 订单的表单对象
const formData = reactive<OrderForm>({});
// 订单表单的校验规则
const rules = reactive({
  name: [{ required: true, message: "请输入角色名称", trigger: "blur" }],
  code: [{ required: true, message: "请输入角色编码", trigger: "blur" }],
});

/**
 * 打开表单弹窗
 */

```

```

function openDialog(): void {
  dialogState.visible = true;
}

/**
 * 关闭表单弹窗
 */
function closeDialog(): void {
  dialogState.visible = false;
  resetForm();
}

/**
 * 重置表单数据和验证状态
 */
function resetForm(): void {
  // 1. 重置orderFormRef 表单所有字段
  orderFormRef.value?.resetFields();
  // 2. 重置orderFormRef 表单所有校验
  orderFormRef.value?.clearValidate();

  // 3. 设置 清空业务字段
  formData.id = undefined;
  formData.name = undefined;
  formData.code = undefined;
  formData.createTime = undefined;
  formData.updateTime = undefined;
  // 4. 设置[下拉列表\复选框\单选按钮组] 的默认值
}

/**
 * 新增按钮点击事件
 */
async function handleCreateClick(): Promise<void> {
  dialogState.title = "新增订单";
  openDialog();
}

/**
 * 编辑按钮点击事件
 * @param roleId 角色ID
 */
async function handleEditClick(id: string): Promise<void> {
  // 1. 设置 弹出窗口 的标题
  dialogState.title = "修改订单";
  // 2. 根据当记录行id 查询要修改的订单数据
  const data = await OrderAPI.getFormData(id);
  // 3. 复制 修改的行数据 到 表单对象上
  Object.assign(formData, data);
  // 4. 打开 弹出窗口
  openDialog();
}

// 提交角色表单
async function handleSubmit(): Promise<void> {
  // 1. 判断表单所有的值是否通过校验
  const valid = await orderFormRef.value?.validate().then(
    () => true,
    () => false
  );
  // 2. 如果没有通过校验 则直接退出提交事件
  if (!valid) return;

  // 3. 如果通过校验 则运行下面代码

  // 3.1 解构 formData 的数据 赋值到 submitData 中
  const submitData = { ...formData };
  loading.value = true;
  try {
    const id = formData.id;
    if (id) {
      await OrderAPI.update(submitData);
      ElMessage.success("修改成功");
    } else {
      await OrderAPI.create(submitData);
      ElMessage.success("新增成功");
    }
    closeDialog();
    handleResetQuery();
  } finally {
    loading.value = false;
  }
}

// 组件挂载后 重置查询条件为第一页 并且 加载订单列表数据
onMounted(() => {
  handleQuery();
});
</script>

<style lang="sass"></style>

```

搜索区域 - 按条件模糊查询

1.修改API接口的类型定义

修改 `src\api\order\order\types.ts` 文件 把OrderQueryParams 的 `keywords` 修改为 `name` 字段

```
/**
 * Order 订单模块 类型定义
 */

// 分页查询条件 - 基础服务类型
import type { BaseQueryParams } from "@api/common";

/** Order 订单分页查询参数 - 分页查询 */
export interface OrderQueryParams extends BaseQueryParams {
  /** 搜索关键字 */
  name?: string;
}

/** 订单分页对象 - 列表查询 */
export interface OrderItem {
  /** 订单ID */
  id?: string;
  /** 订单编码 */
  code?: string;
  /** 订单名称 */
  name?: string;
  /** 创建时间 */
  createTime?: Date;
  /** 修改时间 */
  updateTime?: Date;
}

/** 订单表单对象 - 新增/修改 */
export interface OrderForm {
  /** 角色ID */
  id?: string;
  /** 订单编码 */
  code?: string;
  /** 订单名称 */
  name?: string;
  /** 创建时间 */
  createTime?: Date;
  /** 修改时间 */
  updateTime?: Date;
  /** 自定义数据权限部门ID列表(当dataScope=5时有效) */
  // deptIds?: string[];
}
```

2.编写搜索区域的代码

`queryParams.name` 对应上一步修改的 OrderQueryParams 的 `name` 字段。

```
<el-card class="page-search" shadow="never">
  <el-form ref="queryFormRef" :model="queryParams" :inline="true">
    <el-form-item prop="name" label="订单名称">
      <el-input
        v-model="queryParams.name"
        placeholder="订单名称"
        clearable
        @keyup.enter="handleQuery"
      />
    </el-form-item>

    <el-form-item>
      <el-button type="primary" icon="search" @click="handleQuery">搜索</el-button>
      <el-button icon="refresh" @click="handleResetQuery">重置</el-button>
    </el-form-item>
  </el-form>
</el-card>
```

